honorapp — Especificación del modelo de datos v1

Oct 16, 2026 · @Grafica Dnews

Alcance v1 y principios

La v1 crea todas las tablas, pero solo construye pantallas para pacientes, sesiones, pagos, deudas y resumen mensual con export. Comprobantes, consentimientos y obras sociales quedan sin interfaz hasta la v1.1.

Stack: Supabase (Postgres + Supabase Auth: email y contraseña, recuperación por email, MFA opcional) y deploy en Vercel. Framework del front: pendiente de definir. Solo online, sin modo offline.

Principios

- Una cuenta por profesional, con datos aislados. Todas las tablas llevan `owner_id` directo para que cada política RLS sea una comparación simple.
- Plata en `numeric(14,2)` + `moneda` (`ARS` | `USD`). Nunca `float` .
- Lo que se debe (cargo) se separa de lo que se paga (pago). La relación entre ambos es una aplicación muchos-a-muchos.
- Nada contable se borra: se anula con `anulado_at` . Un monto aplicado es inmutable; para corregir se anula y se crea de nuevo.
- El tipo de cambio se congela en cada pago. Cambiar la configuración nunca altera el pasado.
- Toda escritura sobre tablas financieras pasa por funciones RPC. El cliente lee directo con RLS, pero no escribe directo.
- Todo criterio fiscal queda configurable o pendiente de validación del contador. Nada fiscal se fija en el código.

Tablas, campos y relaciones

Son 16 tablas. Todas llevan además `id uuid` (PK), `owner_id uuid not null default auth.uid()` y `created_at timestamptz default now()` , salvo `profesionales` , cuyo `id` es el de `auth.users` . Toda FK hacia otra tabla del usuario es compuesta con `owner_id` (ver Seguridad). `?` = puede ser nulo.

Tabla	Campos	UI v1
`profesionales`	`nombre`, `tc_fuente_default`, `zona_horaria` (default `America/Argentina/Buenos_Aires`)	Sí (perfil)
`pacientes`	`nombre_visible` (acepta alias o iniciales), `telefono?`, `email?`, `financiador_id?`, `nro_afiliado?`, `archivado_at?`, `suprimido_at?`	Sí
`aranceles`	`paciente_id`, `monto`, `moneda`, `vigente_desde` (date). Unique `(paciente_id, vigente_desde)`	Sí
`agendas`	`paciente_id`, `frecuencia` (`semanal` | `quincenal` | `mensual`), `dia_semana` (1–7), `hora` (time), `semana_del_mes?` (1–5, solo mensual), `fecha_ancla` (date, define las semanas de la quincenal), `vigente_desde`, `vigente_hasta?`	Sí
`financiadores`	`nombre`, `tipo` (`obra_social` | `prepaga` | `otro`), `activo`	No (v1.1)
`consentimientos`	`paciente_id`, `version_texto`, `medio`, `otorgado_at`, `revocado_at?`	No (v1.1)
`prestaciones`	`paciente_id`, `fecha` (date), `hora?`, `concepto`, `estado` (`realizada` | `ausente_cobrable` | `ausente_no_cobrable` | `cancelada`), `agenda_id?`, `importacion_id?`	Sí (sesiones)
`cargos`	`prestacion_id?`, `concepto`, `deudor_tipo` (`paciente` | `financiador`), `paciente_id?`, `financiador_id?`, `monto`, `moneda`, `fecha_vencimiento?`, `anulado_at?`, `anulado_motivo?`	Indirecta
`pagos`	`fecha_cobro`, `pagador_tipo`, `paciente_id?`, `financiador_id?`, `monto`, `moneda`, `medio`, `referencia?`, `tc_valor`, `tc_fuente`, `tc_fecha`, `monto_ars` (generada), `importacion_id?`, `anulado_at?`, `anulado_motivo?`, `reemplaza_a?`	Sí
`imputaciones`	`pago_id`, `cargo_id`, `monto_en_moneda_cargo`, `monto_en_moneda_pago`, tc_aplicado?, `origen` (`auto` | `manual`), `anulado_at?`	Avanzada

Tabla	Campos	UI v1
`ajustes`	`cargo_id`, `monto`, `motivo` (`debito_obra_social` | `bonificacion` | `incobrable`), `nota?`, `fecha`, `anulado_at?`	Sí (en deudas)
`comprobantes`	`tipo`, `punto_venta`, `numero`, `fecha_emision`, `receptor_tipo`, `paciente_id?`, `financiador_id?`, `monto_ars`, `cae?`, `anulado_at?`. Unique `(owner_id, tipo, punto_venta, numero)`	No (v1.1)
`comprobante_vinculos`	`comprobante_id`, `pago_id?` o `cargo_id?` (exactamente uno), `monto_ars`, `anulado_at?`	No (v1.1)
`tipos_cambio`	`fecha`, `fuente` (texto libre), `valor`. Unique `(owner_id, fecha, fuente)`	Sí (config)
`importaciones`	`archivo_nombre`, `estado`, `filas_ok`, `filas_error`, `errores` (jsonb), `deshecha_at?`	Sí
`auditoria`	`tabla`, `registro_id`, `accion`, `cambios` (jsonb), `actor`, `at`	No

Checks principales

- `cargos` y `pagos`: exactamente uno de `paciente_id` / `financiador_id`, según el tipo de deudor o pagador.
- `pagos`: si `moneda = 'ARS'`, `tc_valor = 1`. Si es `USD`, los tres campos `tc_*` son obligatorios. `monto_ars = round(monto * tc_valor, 2)`.
- Montos siempre `> 0`.
- `agendas`: `semana_del_mes` obligatorio solo si `frecuencia = 'mensual'`.
- `prestaciones`: unique parcial `(agenda_id, fecha)` donde `agenda_id is not null`. Es lo que vuelve idempotente la generación mensual.

Cambios respecto del modelo aprobado

- `arancel_default_*` sale de `pacientes` y pasa a `aranceles`, con vigencia. Sin historial, generar un mes pasado usaría el arancel de hoy.
- La frecuencia va en `agendas` y no en `pacientes`: cambia con el tiempo y un paciente puede tener dos sesiones semanales.
- `imputaciones`, `ajustes` y `comprobante_vinculos` suman `anulado_at`.
- `prestaciones` suma `hora` y `agenda_id`.

Ciclo de vida: anulaciones, cascadas e inmutabilidad

Toda anulación ocurre en una sola RPC transaccional, y la plata que queda liberada vuelve a aplicarse automáticamente (FIFO) a las deudas abiertas del mismo paciente.

Acción	Efecto en cascada	Bloqueos
`anular_pago(pago_id, motivo)`	Anula sus imputaciones activas. Los cargos afectados vuelven a tener saldo.	Rechaza si el pago tiene vínculos activos a comprobantes. Hay que anular el vínculo primero.
`corregir_pago(pago_id, datos_nuevos)`	Anula el pago y crea uno nuevo con `reemplaza_a = pago_id`, aplicado por FIFO.	Las mismas que `anular_pago`.
`cambiar_estado_prestacion(id, estado)` a `cancelada` o `ausente_no_cobrable`	Anula sus cargos, sus imputaciones y sus ajustes. La plata liberada queda como saldo a favor y se reaplica por FIFO.	Rechaza si algún cargo tiene vínculo activo a comprobante.
`cambiar_estado_prestacion` a `realizada` o `ausente_cobrable` desde un estado sin cargo	Crea el cargo con el arancel vigente a la fecha de la prestación y aplica saldo a favor.	Si no hay arancel vigente, crea la prestación sin cargo y devuelve una advertencia.
`registrar_ajuste(cargo_id, monto, motivo)`	Reduce el saldo del cargo.	Rechaza si `monto` > saldo actual del cargo.
`anular_ajuste(ajuste_id)`	El cargo recupera ese saldo.	Ninguno.
`anular_imputacion(id)` (avanzada)	El pago recupera saldo a favor. No se reaplica automáticamente: el usuario lo está asignando a mano.	Ninguno.

Inmutabilidad

- `pagos`: `monto`, `moneda`, `tc_*`, `fecha_cobro`, `paciente_id` y `financiador_id` no se pueden editar si hay imputaciones o vínculos activos. Como la aplicación FIFO es automática, en la práctica casi todo pago queda bloqueado al registrarse. Por eso

existe `corregir_pago`: en la interfaz aparece como "Editar" y por detrás anula y recrea.

- `cargos`: `monto`, `moneda` y deudor bloqueados si hay imputaciones o ajustes activos.
- `comprobantes`: `monto_ars` bloqueado si hay vínculos activos.
- Campos libres (`referencia`, `nota`, `concepto`, `medio`) se pueden editar siempre, con registro en auditoría.
- Un registro anulado no se reactiva. Para volver atrás se crea uno nuevo.
- Se implementa con un trigger `before update` por tabla, no solo en la RPC, para que un error futuro en el código no lo saltee.

Concurrencia: las RPCs bloquean con `select … for update` las filas de pagos y cargos involucradas antes de validar saldos. Sin eso, dos operaciones simultáneas pueden pasar el control y aplicar de más.

Agenda recurrente y generación de sesiones del mes

`generar_prestaciones_mes(anio, mes, paciente_ids?)` crea en bloque las sesiones de cada agenda vigente con su cargo, y se puede correr varias veces sin duplicar.

Reglas de fechas

- `semanal`: todos los `dia_semana` del mes.
- `quincenal`: los `dia_semana` que caen a múltiplos de 14 días desde `fecha_ancla`.
- `mensual`: el N-ésimo `dia_semana` del mes (`semana_del_mes`; 5 = el último). Ejemplo: "primer martes". *Decisión tomada por defecto; si preferís "día 15 de cada mes", hay que cambiar el campo.*
- Solo fechas dentro de `vigente_desde` / `vigente_hasta` de la agenda y con el paciente no archivado.

No duplicar

1. Si ya existe una prestación con esa `(agenda_id, fecha)`, se saltea, **en cualquier estado**. Una sesión cancelada a mano no se recrea al volver a generar.
2. Si el paciente ya tiene otra prestación esa fecha (cargada a mano o importada), también se saltea.
3. El unique parcial `(agenda_id, fecha)` garantiza lo anterior aunque dos ejecuciones corran a la vez.

Cargos y saldo a favor

- Cada sesión creada recibe un cargo al paciente con el arancel vigente a su fecha (ver Aranceles).

- Sin arancel vigente: la sesión se crea sin cargo y el paciente aparece en las advertencias del resultado.
- Al terminar, se aplica por FIFO el saldo a favor que tenga cada paciente. Así un pago adelantado cubre las sesiones nuevas solo.
- En v1 no hay cargos a obras sociales desde la generación, porque no tienen interfaz.

Resultado: devuelve creadas, salteadas (con motivo) y advertencias, para mostrar un resumen antes de cerrar.

Sesiones futuras: la generación crea sesiones con fecha posterior a hoy, en estado `realizada`. El usuario solo marca las excepciones (ausencias, cancelaciones). Las vistas de deuda cuentan solo cargos de sesiones con fecha ⩽ hoy, así una sesión del día 28 no figura como deuda el día 5. La interfaz muestra las futuras como "programadas".

Fuera de v1: feriados. La generación no los conoce; el usuario cancela esas sesiones a mano.

Aranceles con vigencia y actualización masiva

El arancel vigente a una fecha es la fila de `aranceles` con el mayor `vigente_desde` ⩽ esa fecha. Nunca se pisa una fila: un aumento es una fila nueva.

`actualizar_arancel(paciente_ids[], monto, moneda, desde, recalcular_existentes)`

- Inserta una fila por paciente con `vigente_desde = desde`. Si ya existe una con esa fecha, la reemplaza (es una corrección del mismo aumento).
- Si `recalcular_existentes = true`: los cargos de sesiones con fecha ⩾ `desde` **sin imputaciones ni ajustes activos** se anulan y se recrean con el nuevo monto, y se aplica FIFO.
- Los cargos que ya tienen plata aplicada no se tocan. Se devuelven como "omitidos" para que el usuario decida (por ejemplo, registrar la diferencia como un cargo aparte).
- Variante de interfaz: "Aumentar X %" calcula el monto nuevo de cada paciente a partir de su arancel vigente y llama a la misma RPC. El redondeo se decide en la interfaz antes de llamar.

Por qué no se recalculan los cargos con pagos aplicados: si una sesión ya está paga al precio anterior, cambiarle el monto genera deuda retroactiva sin que el profesional lo decida.

Aplicación automática de pagos (FIFO) y ajuste manual

Por defecto, todo pago de un paciente se aplica solo a sus deudas más viejas primero; lo que sobra queda como saldo a favor y se usa cuando aparezcan cargos nuevos.

`registrar_pago(datos, asignacion = 'auto' | 'manual', items?)`

Orden FIFO

1. Cargos activos del mismo paciente con saldo > 0, incluidos los de sesiones futuras (así funciona el pago adelantado).
2. Ordenados por fecha de la sesión; los cargos sin sesión, por `fecha_vencimiento` o `created_at`. Desempate por `created_at`.
3. Del lado de los pagos, si hay varios con saldo a favor, se usa primero el más viejo.
4. Nunca toca imputaciones existentes: solo reparte plata no aplicada.

Cuándo corre automáticamente: al registrar un pago, al crear cargos (generación mensual, sesión suelta, recálculo de aranceles) y al liberar plata por una anulación. No corre después de `anular_imputacion`, porque ahí el usuario está asignando a mano.

Monedas distintas (arancel en USD, pago en ARS, o al revés)

- Se necesita un tipo de cambio para la conversión. Se toma de `tipos_cambio` con la fuente default del profesional, a la fecha del cobro o la última anterior. Se guarda en `imputaciones.tc_aplicado`.
- Si no hay cotización cargada, no se aplica entre monedas: el pago queda como saldo a favor y la RPC devuelve una advertencia.
- El cálculo se hace en la moneda del cargo y se redondea a 2 decimales. Si el saldo restante del cargo queda por debajo de 0,01, el cargo se considera pagado.
- *Cuál cotización usar lo define el contador (ver Pendientes).*

Modo manual (opción avanzada): el usuario elige los cargos y los montos. Lo no asignado queda como saldo a favor y no se reparte automáticamente hasta el próximo disparador.

Obras sociales: en v1 no hay pagos de financiadores desde la interfaz. Cuando lleguen (v1.1), irán por defecto en modo manual, porque una liquidación detalla qué sesiones cubre.

Valores calculados y vistas

Ningún saldo, estado ni total se guarda: todo sale de vistas creadas con `security_invoker = true`, porque sin esa opción una vista ignora RLS y muestra datos de todos los usuarios.

Vista	Qué calcula	Pantalla v1
`v_cargos`	Saldo = monto − imputado activo − ajustes activos. Estado: `pendiente` | `parcial` | `pagado` | `ajustado` | `anulado`. Marca `es_futuro` si la sesión es posterior a hoy.	Deudas
`v_pagos`	Aplicado y saldo a favor de cada pago.	Pagos
`v_deuda_paciente`	Por paciente y por moneda: deuda vencida (solo sesiones ≤ hoy), próximas sesiones a cobrar y saldo a favor.	Deudas, pacientes
`v_estado_mes_paciente`	Por paciente y mes: `al_dia` | `debe` | `parcial` | `sin_sesiones`. Reemplaza el "pagó / no pagó".	Resumen mensual
`v_resumen_mensual`	Sesiones realizadas, ausencias cobrables y no cobrables, cancelaciones, pacientes atendidos (distintos), cobrado por moneda, cobrado en ARS, generado en el mes, pendiente del mes.	Resumen mensual
`v_cobros_sin_facturar`	Pagos con `monto_ars` mayor a lo vinculado a comprobantes.	v1.1
`v_ingresos_12m`	Acumulado móvil de 12 meses en dos versiones: cobrado y facturado.	v1.1

Criterios

- "El mes" se calcula en la zona horaria del profesional. Las sesiones van por `fecha` de la prestación; lo cobrado, por `fecha_cobro`.
- ARS y USD nunca se suman en una misma cifra. Un total en pesos de deudas en dólares aparece solo como referencia, con la cotización usada a la vista.
- Lo anulado no cuenta en ningún cálculo.

Export para el contador (v1)

- Formatos: CSV y XLSX.
- Hoja 1, resumen del mes: los totales de `v_resumen_mensual`.
- Hoja 2, detalle de cobros: fecha, paciente (`nombre_visible`), monto, moneda, cotización y fuente, monto en ARS, medio, referencia.
- Hoja 3, deudas al cierre del mes, por paciente y moneda.

- Opción de exportar con iniciales en lugar del nombre visible, para minimizar los datos que salen de la app.
- *Qué columnas necesita el contador para su liquidación hay que confirmarlo con él.*

Seguridad: RLS, RPCs y auditoría

El cliente lee todo con RLS, pero solo escribe directo en tablas sin plata; todo lo financiero se escribe por RPC, que es donde viven las cascadas, los bloqueos y el FIFO.

Tabla	Lectura	Escritura del cliente	Delete
`profesionales`	RLS (`id = auth.uid()`)	Update propio	No
`pacientes`	RLS	Insert / update	No (archivar o suprimir por RPC)
`agendas`	RLS	Insert / update	No (se cierra con `vigente_hasta`)
`tipos_cambio`	RLS	Insert / update / delete	Sí (los valores ya se copiaron a cada pago)
`financiadores` , `consentimientos`	RLS	Insert / update	No
`aranceles` , `prestaciones` , `cargos` , `pagos` , `imputaciones` , `ajustes` , `comprobantes` , `comprobante_vinculos` , `importaciones`	RLS	Ninguna: solo RPC	No
`auditoria`	RLS	Ninguna: solo triggers	No

Políticas RLS

- `enable row level security` en las 16 tablas, sin excepción.
- Patrón: `using (owner_id = (select auth.uid()))` y el mismo `with check` en insert y update.
- Si el usuario activó MFA, las políticas exigen sesión `aal2` (patrón de Supabase), así el segundo factor protege los datos y no solo el login.

- Índice en `owner_id` en cada tabla.

RPCs

- `security definer` con `set search_path = ''` y nombres de tabla calificados. Como estas funciones saltean RLS, cada consulta interna filtra explícitamente por `owner_id = auth.uid()`. Los tests de RLS cubren cada RPC.
- Se revoca `insert`, `update` y `delete` a `authenticated` sobre las tablas de solo-RPC. Se revoca `execute` a `anon` sobre todas las funciones.
- La clave `service_role` nunca llega al navegador; en Vercel queda solo en variables del servidor, si se usa.

Integridad entre usuarios

- Las foreign keys no respetan RLS: sin protección extra, un usuario podría crear un cargo propio apuntando al paciente de otro. Por eso cada tabla padre tiene `unique (id, owner_id)` y cada FK es compuesta: `foreign key (paciente_id, owner_id) references pacientes (id, owner_id)`.
- `owner_id` es inmutable: trigger que rechaza su cambio en todas las tablas.

Auditoría: sí hace falta

Las anulaciones dejan rastro solas, pero las ediciones de campos libres y quién ejecutó cada RPC no. Diseño:

- Trigger `after insert or update` en `pacientes`, `aranceles`, `agendas`, `prestaciones`, `cargos`, `pagos`, `imputaciones`, `ajustes`, `comprobantes` y `comprobante_vinculos`.
- `cambios` guarda solo las columnas que cambiaron, con valor anterior y nuevo.
- Para datos personales (`nombre_visible`, `telefono`, `email`, `nro_afiliado`) se registra que cambiaron, **sin los valores**. Si no, la auditoría guardaría una copia de los datos que la supresión tiene que borrar.
- `actor = auth.uid()`. El usuario ve su propio historial por RLS y no puede modificarlo.
- Retención: a definir con el contador y el abogado.

Supresión de datos de pacientes

Suprimir un paciente lo anonimiza y conserva sus registros contables; es una propuesta a validar con el abogado.

`suprimir_paciente(paciente_id)`

1. Borra `telefono`, `email` y `nro_afiliado`.
2. Cambia `nombre_visible` a "Paciente suprimido" + un número corto.

3. Marca `suprimido_at` y revoca sus consentimientos activos.
4. Cierra sus agendas (`vigente_hasta = hoy`) y cancela sus sesiones futuras.
5. Deja intactos cargos, pagos y ajustes, ya anonimizados por el paso 2.
6. No se puede deshacer.

Las filas de `auditoria` del paciente no contienen datos personales (ver Seguridad), así que no hace falta tocarlas. Los exports ya descargados quedan fuera del alcance de la app.

Vocabulario de la interfaz

Los nombres internos no aparecen en pantalla; la palabra "imputación" no se muestra nunca.

Interno	En pantalla
prestación	sesión
prestación futura	sesión programada
cargo	a cobrar / debe
imputación	"aplicado a" ("Pago aplicado a 3 sesiones")
saldo no aplicado de un pago	saldo a favor
asignación manual	"Elegir qué sesiones cubre"
ajuste	descuento / bonificación / no se cobra
anular	anular (con motivo)
`corregir_pago`	editar pago
`ausente_cobrable` / `ausente_no_cobrable`	ausente, se cobra / ausente, no se cobra
tipo de cambio	cotización
suprimir	eliminar datos del paciente

Pendientes de validación

Nada de esta lista está verificado en fuentes oficiales al 16/10/2026; cada punto lo tiene que confirmar un profesional matriculado antes de implementarse.

Contador

- [ ] Qué cotización usar para convertir USD a ARS (fuente y tipo: comprador, vendedor, otra) y a qué fecha.
- [ ] Criterio de facturación: por cobro o por prestación. Define cómo se usan `comprobante_vinculos`.
- [ ] Tipo de comprobante que corresponde a cada profesional.
- [ ] Qué base de ingresos se usa para la recategorización del monotributo (cobrado, facturado u otra).
- [ ] Columnas y formato del export que necesita para su trabajo.
- [ ] Plazo de conservación de los registros contables y de la auditoría.

Abogado (datos personales)

- [ ] Si hace falta consentimiento del paciente y en qué forma (define el uso de `consentimientos`).
- [ ] Transferencia internacional a Brasil (Supabase San Pablo): si requiere cláusulas contractuales o consentimiento.
- [ ] Quién es responsable y quién encargado del tratamiento: la app o cada profesional. Afecta términos y condiciones.
- [ ] Si corresponde inscribir la base ante la AAIP.
- [ ] Si la supresión por anonimización cumple con el derecho de supresión.
- [ ] Si hubo cambios en la ley de datos personales vigentes a la fecha.

Producto

- [ ] Framework del front.
- [ ] Agenda mensual: N-ésimo día de la semana (decisión actual) o día fijo del mes.